Memòria de la Pràctica: Mr. J System



Sistemes Operatius

Laura Cuenca – Laura.cuenca@students.salle.url.edu

Curs 2024-25

Índex

1. Introducció 3

2. Disseny 4

2.1. Arquitectura general 4

2.2. Estructures de Dades 4

2.3. Recursos del Sistema 4

Sockets TCP/IP 4

Threads 5

Mutex 6

Altres mecanismes 6

2.4. Protocol de comunicació 7

3. Implementació per fases 8

3.1. Fase 1: Configuració i Shell 8

3.2. Fase 2: Connexions 8

3.3. Fase 3: Distorsió de fitxers multimèdia 9

3.4. Fase 4: Distorsió de text i registres 9

4. Problemes observats i solucions 10

5. Diagrames de Flux 13

Connexió inicial de Fleck amb Gotham i dels Workers amb Gotham 13

Fleck 13

Workers 14

Gestió de Petició de Distorsió 14

6. Estimació temporal 16

7. Conclusions i propostes de millora 17

Conclusions sobre la feina feta 17

Propostes de millora per a la propera entrega 18

8. Bibliografia 22

1. Introducció

El projecte *Mr. J System* ha consistit en el disseny i la implementació d'un sistema distribuït que permet gestionar la distorsió de fitxers de text i mitjans audiovisuals, basant-se en una arquitectura client-servidor robusta i escalable. Els components principals del sistema són:

- **Gotham**: Servidor central que coordina les connexions i distribueix tasques.
- **Fleck**: Clients que fan peticions de distorsió.
- **Harley**: Processos per distorsionar fitxers multimèdia.
- **Enigma**: Processos per distorsionar fitxers de text.
- **Arkham**: Subprocés de Gotham encarregat del registre d'esdeveniments.

Pel que fa a la temàtica de **Mr. J. System**, aquesta pràctica està inspirada en l'univers de **Batman**, concretament en el personatge del Joker (Arthur Fleck) i altres figures rellevants com Harley Quinn i l'Enigma (Riddler). Aquesta relació es desprèn de diversos elements:

- **Arthur Fleck** és el nom original del Joker en la pel·lícula *Joker* (2019). Aquest és el personatge central del sistema, que busca implementar un món amb la seva "realitat distorsionada".
- **Harley** i **Enigma** són els noms dels *workers* encarregats de processar peticions de distorsió de *media* i text, respectivament. Aquests noms fan referència a Harley Quinn i al Riddler, personatges icònics de l'univers de Batman.
- **Gotham**, el servidor central, representa la ciutat fictícia de l'univers de Batman. Aquest servidor és el cor del sistema i actua com a punt de connexió per als clients (Fleck) i els *workers*.

2. Disseny

2.1. Arquitectura general

La implementació segueix una arquitectura distribuïda basada en sockets TCP/IP, amb els següents components:

- **Gotham**:
  - Registra connexions entrants.
  - Coordina tasques entre clients i workers.
  - Redistribueix treballs en cas de fallades.
- **Fleck**:
  - Gestiona les connexions amb Gotham i els workers.
  - Permet a l'usuari sol·licitar distorsions i supervisar-ne l'estat.
- **Harley i Enigma**:
  - Processen fitxers multimèdia i de text, respectivament.
  - Implementen mecanismes de recuperació per garantir robustesa.
- **Arkham**:
  - Registra esdeveniments i errors en temps real en un fitxer únic (logs.txt).

2.2. Estructures de Dades

Les estructures principals utilitzades són:

1. **ConfigBase**: Guarda informació bàsica de configuració (IP, port, tipus d'instància). aquestes dades facilita passar paràmetres entre funcions de configuració i connexió.
2. **Task**: Representa tasques pendents dels workers, agrupació necessària per gestionar múltiples connexions concurrents en workers.
3. **Trama**: Defineix el protocol de comunicació entre processos, incloent camps com **checksum** i **timestamp** per garantir la integritat entre els diferents .c.

2.3. Recursos del Sistema

Sockets TCP/IP

Els **sockets TCP/IP** han estat triats com a mecanisme principal de comunicació per diverses raons:

- **Requisit del projecte:** La pràctica requereix la comunicació entre processos situats en màquines distribuïdes. Altres mecanismes com **pipes** o **memòria compartida** són limitats a comunicacions dins de la mateixa màquina i no són adequats per a sistemes distribuïts.
- **Connexions fiables:** Els sockets orientats a connexió (TCP) garanteixen una transmissió fiable, amb mecanismes automàtics per a la detecció d'errors, retransmissions i ordre dels paquets. Això els fa més adequats que sockets **UDP**, que no garanteixen la integritat ni l'ordre dels missatges.
- **Escalabilitat:** Els sockets TCP/IP són una opció robusta per escenaris amb múltiples clients connectant-se simultàniament al servidor. En comparació amb altres mecanismes de comunicació, com les **cues de missatges**, els sockets no requereixen còpies addicionals de dades i són més eficients en termes de transferència de volum de dades grans.

Threads

S'han utilitzat **threads** per gestionar connexions simultànies amb múltiples clients, prioritzant el rendiment i l'eficiència:

- **Rendiment:** Els threads són més lleugers que els processos creats amb **fork()**, ja que comparteixen la mateixa memòria i recursos del procés principal. Això redueix la sobrecàrrega associada a la creació de nous espais d'adreces.
- **Complexitat de gestió:** Amb **fork()**, cada procés fill requereix mecanismes addicionals per compartir dades amb el procés pare o altres processos (com **memòria compartida** o **pipes**). En canvi, els threads poden accedir directament a les dades globals del procés principal, simplificant la comunicació interna.
- **Escenaris amb molts clients:** A diferència de **fork()**, que pot esgotar els recursos del sistema quan hi ha molts clients simultanis, els threads tenen un menor cost de creació i gestió, fent-los ideals per a aplicacions amb un alt nombre de connexions.

Mutex

S'han utilitzat en Harley per evitar condicions de carrera en l'accés a la llista de tasques. Els mutex són una eina senzilla i directa per garantir l'exclusió mútua en l'accés a recursos compartits entre threads. Això els fa més adequats que altres mecanismes com **semafòrs** quan l'objectiu principal és protegir una secció crítica.

Altres mecanismes

En aquesta pràctica s'han prioritzat els sockets i els threads com a mecanismes principals de comunicació i gestió, descartant altres opcions en funció dels requisits específics i de l'eficiència necessària. Tot i això, es considera la implementació de la **memòria compartida** per a certes funcionalitats.

Les **pipes** són un mecanisme de comunicació unidireccional limitat a processos dins de la mateixa màquina. Això les fa inadequades per a sistemes distribuïts, que és un requisit essencial en aquesta pràctica. A més, no són eficients en escenaris amb múltiples clients, ja que la seva gestió es complica per la necessitat de manejar múltiples descriptors de fitxers.

Les **cues de missatges**, tot i ser útils per proporcionar sincronització implícita, són més lentes que els sockets per a grans volums de dades, a causa de les còpies addicionals necessàries durant la transmissió, a més no són utilitzables en sistemes distribuits, si més no amb els coneixements que disposo actualment.

Pel que fa a la **memòria compartida** es valora el seu ús futur per gestionar les tasques entre **workers del mateix tipus dins d'una mateixa màquina**. Aquesta opció permet compartir dades de manera eficient entre processos, encara també requereix l'ús addicional de mecanismes com **semafòrs** o **mutex**.

Finalment, utilitzar **fork** en lloc de threads ha estat descartat perquè implicaria la duplicació completa del procés pare per cada connexió, cosa que no és escalable en entorns amb molts clients. A més, això requeriria recursos addicionals i mecanismes més complexos, com memòria compartida o pipes, per compartir informació entre processos fills i el procés pare.

2.4. Protocol de comunicació

S'han utilitzat trames serialitzades de 256 bytes amb els camps següents:

- **Type**: Tipus de trama.
- **Data Length**: Longitud de les dades.
- **Data**: Informació de la trama.
- **Checksum**: Validació d'integritat.
- **Timestamp**: Temps d'enviament.

3. Implementació per fases

3.1. Fase 1: Configuració i Shell

En aquesta primera fase, es va treballar en la lectura de fitxers de configuració i la implementació del Shell de Fleck. Cada procés (Gotham, Harley, Enigma i Fleck) llegeix la seva configuració inicial des de fitxers, garantint una inicialització adequada i flexible. Es va dissenyar un Shell per Fleck que permet executar comandes bàsiques com CONNECT, LOGOUT, LIST MEDIA i LIST TEXT, proporcionant una interfície funcional per als usuaris.

Per facilitar la gestió del codi i promoure la reutilització, es va optar per separar les funcions relacionades amb la configuració en mòduls independents, com ara file_utils.c. També es van introduir dues estructures fonamentals: ConfigBase, per unificar la gestió de configuracions dels diferents components, i Trama, que facilita la comunicació entre processos amb un format consistent.

3.2. Fase 2: Connexions

Durant aquesta fase, es va establir la base per a les comunicacions entre els diferents components del sistema. Es van implementar connexions estables entre Gotham i Fleck, amb Gotham actuant com a servidor intermediari. Aquest component s'encarrega de redirigir les peticions de distorsió cap als **workers** corresponents, assegurant que el sistema funcioni de manera integrada.

La decisió de disseny principal en aquesta fase va ser l'ús de **sockets TCP/IP**, que proporcionen un canal robust i fiable per a les comunicacions entre nodes distribuïts. També es va implementar una gestió inicial de fallades: en cas que un **worker** caigui, Gotham reasigna automàticament la tasca a un altre **worker** disponible mitjançant trames de tipus 0x11. Aquesta funcionalitat assegura la continuïtat del servei i redueix l'impacte de possibles errors.

3.3. Fase 3: Distorsió de fitxers multimèdia

En aquesta etapa, es va incorporar la funcionalitat de distorsió de fitxers multimèdia, amb Harley utilitzant la llibreria SO_compression.h per comprimir imatges i àudios. A més, es va afegir la capacitat de recuperació automàtica de tasques en cas de fallades d'un **worker**, millorant la robustesa global del sistema.

Es va prioritzar l'ús de memòria dinàmica per optimitzar el rendiment i evitar limitacions per la gestió fixa de memòria. També es van implementar **threads** en els **workers** per gestionar múltiples clients simultàniament. Això permet que cada connexió funcioni de manera independent, garantint la flexibilitat i escalabilitat del sistema. Aquesta decisió va ser clau per aconseguir un alt rendiment en entorns amb càrregues elevades.

3.4. Fase 4: Distorsió de text i registres

En l'última fase, es va afegir la funcionalitat de distorsió de text, gestionada per Enigma, que elimina paraules curtes segons un llindar predefinit. A més, es va implementar Arkham, un sistema centralitzat per al registre d'esdeveniments, que guarda logs en un fitxer (logs.txt). Aquest sistema és essencial per monitoritzar i depurar el comportament del sistema.

Per garantir la consistència en entorns concurrents, es va utilitzar un **mutex** per protegir l'accés al fitxer de logs. Això assegura que només un procés pugui escriure-hi alhora, evitant corrupcions.

4. Problemes observats i solucions

- **Desenvolupament en Solitari**

  Aquesta pràctica està pensada per ser desenvolupada en parelles, però el meu company va deixar de respondre poc abans de l'entrega del checkpoint per la fase 2. Això va implicar que hagués d'assumir tota la càrrega de treball i presa de decisions en solitari, incrementant significativament la complexitat del procés. Vaig redistribuir les tasques per complir els objectius essencials dins el temps disponible. Tot i que vaig aconseguir completar les fases requerides, la manca de suport va limitar el temps disponible per al testing avançat i les millores opcionals. Això també va augmentar la pressió durant el desenvolupament.

- **Mala reconstrucció del fitxer per lectura incorrecta de trames**

  Un problema crític va afectar la reconstrucció dels fitxers per part dels workers: el contingut rebut era incomplet o malmès. El problema es devia al fet que les trames llegides contenien més bytes dels realment útils, ja que es feia servir tota la mida de la struct en lloc de limitar-se als bytes indicats pel camp data_length.

  Aquesta gestió incorrecta generava inconsistències en la lectura, trames amb informació residual, errors en la validació de checksum i fallades en la interpretació del contingut (com noms de fitxer mal formats o valors de control incorrectes).

  Per solucionar-ho, es va revisar el procés de recepció i manipulació de trames: es garanteix que sempre es rebin trames completes de mida fixa, però només s'utilitzen els bytes vàlids indicats per data_length. També es va assegurar que totes les operacions sobre les dades fossin segures i consistents, permetent una reconstrucció correcta i fiable del fitxer.

- **Saturació del sistema per excés de processos simultanis**

  Durant les primeres proves amb peticions concurrents, el sistema Montserrat es trobava amb bloquejos i fallades greus. El problema principal era la creació descontrolada de fils dins dels Workers, especialment quan es rebien múltiples trames en paral·lel. Això generava una saturació dels recursos del sistema, amb un excés de processos i sockets oberts, provocant errors d'execució, pèrdua de respostes i una inestabilitat generalitzada. L'arquitectura inicial no estava preparada per gestionar un volum tan alt de connexions simultànies de manera eficient.

  Per solucionar aquest problema, es va optar per implementar un sistema de thread pool fix de 4 fils per Worker, tant a Harley com a Enigma. Aquesta estratègia permet gestionar múltiples connexions entrants provinents de Fleck de

forma concurrent, evitant la penalització de crear i destruir fils per cada nova petició. En el moment d'inicialitzar el Worker, es generen els quatre fils que executen de manera contínua la funció worker_loop, romanent a l'espera de noves connexions mitjançant una cua protegida amb mutex i variables de condició.

Quan un procés Fleck inicia una petició de distorsió (trama 0x03), el socket associat s'enfila a la cua. Quan un dels fils del pool queda disponible, es desbloqueja, extreu la connexió i la delega a la funció handle_task. Aquesta funció s'encarrega de tot el cicle de vida de la petició: recepció i validació de la trama, recepció del fitxer (trames 0x05), processament de la distorsió (compressió o neteja de contingut), enviament del fitxer distorsionat (trames 0x04 i 0x05) i comprovació final del hash MD5 (trama 0x06).

Quan la tasca es completa, el fil no es destrueix sinó que torna a estar disponible per gestionar una nova connexió, fet que garanteix una reutilització eficient dels recursos. Aquesta solució aporta múltiples avantatges: redueix la càrrega associada a la gestió de fils, permet escalar a un nombre elevat de peticions simultànies, assegura una sincronització segura mitjançant mecanismes de concurrència, i introdueix una cua d'espera que actua com a buffer davant pics puntuals de demanda.

Gràcies a aquest nou disseny basat en thread pools, el sistema Montserrat ha guanyat en estabilitat, escalabilitat i robustesa, estant ara preparat per gestionar entorns amb alta concurrència sense comprometre el rendiment ni la fiabilitat.

- **Bloqueig en l'acceptació de múltiples connexions de Fleck**

  Inicialment, Gotham no gestionava correctament les connexions simultànies dels diferents processos Fleck. El disseny original del listener contenia un bucle que bloquejava l'execució mentre esperava que es completés la connexió amb un únic Fleck abans de continuar.

  Aquesta estructura feia que, mentre s'atendia una connexió existent, no es poguessin acceptar de noves, provocant la pèrdua o retard de les peticions entrants.

  La solució implementada va consistir a separar clarament l'acceptació de connexions i la seva gestió posterior. Així, el listener pot continuar acceptant noves connexions sense esperar que es completin les anteriors, permetent una atenció concurrent i fluida dels Fleck i garantint un comportament correcte del sistema en entorns amb múltiples usuaris actius alhora.

- **Limitació actual en l'execució seqüencial de comandes des de Fleck**

  Tot i les millores implementades en la gestió concurrent dels Workers, el sistema encara presenta una limitació important pel que fa a la comunicació entre Fleck i el Worker. En l'estat actual, Fleck no pot enviar una nova comanda fins que la distorsió de la petició anterior hagi finalitzat completament. Això significa que l'execució de les operacions des de Fleck és, de facto, seqüencial i bloquejant, i obliga el procés a esperar la finalització de cada comanda abans de poder continuar amb la següent.

  Aquesta restricció contradiu una de les especificacions inicials del sistema, que plantejava un funcionament realment asíncron i no bloquejant entre Fleck i el Worker. El problema es va detectar en una fase molt avançada del desenvolupament, quan ja s'havien prioritzat altres aspectes del sistema. A causa de la manca de temps i de la complexitat que hauria suposat refactoritzar la comunicació, es va decidir no implementar l'asimetria esperada en aquest punt.

- **Estructura de configuració i tipus de *workers***

  En treballar amb els mòduls "Harley", "Enigma", "Fleck" i "Gotham", es va identificar que les estructures de configuració inicials tenien redundàncies significatives. Per optimitzar i simplificar el codi, es va dissenyar una estructura genèrica anomenada ConfigBase, que inclou els camps comuns per a tots els tipus d'instàncies, com ara la IP, el port i el descriptor del socket. A partir de ConfigBase, es van crear estructures específiques per als diferents mòduls:

  - ConfigGotham: Afegeix camps per gestionar la IP i el port dels workers que es connecten a "Gotham". Això permet a "Gotham" coordinar instàncies com "Harley" i "Enigma".
  - ConfigWorker: Inclou camps per gestionar el directori de treball i l'estat de disponibilitat dels workers ("Harley" i "Enigma"). Això assegura una configuració adequada per cada worker segons si està disponible o ocupat.

5. Diagrames de Flux

Connexió inicial de Fleck amb Gotham i dels Workers amb Gotham

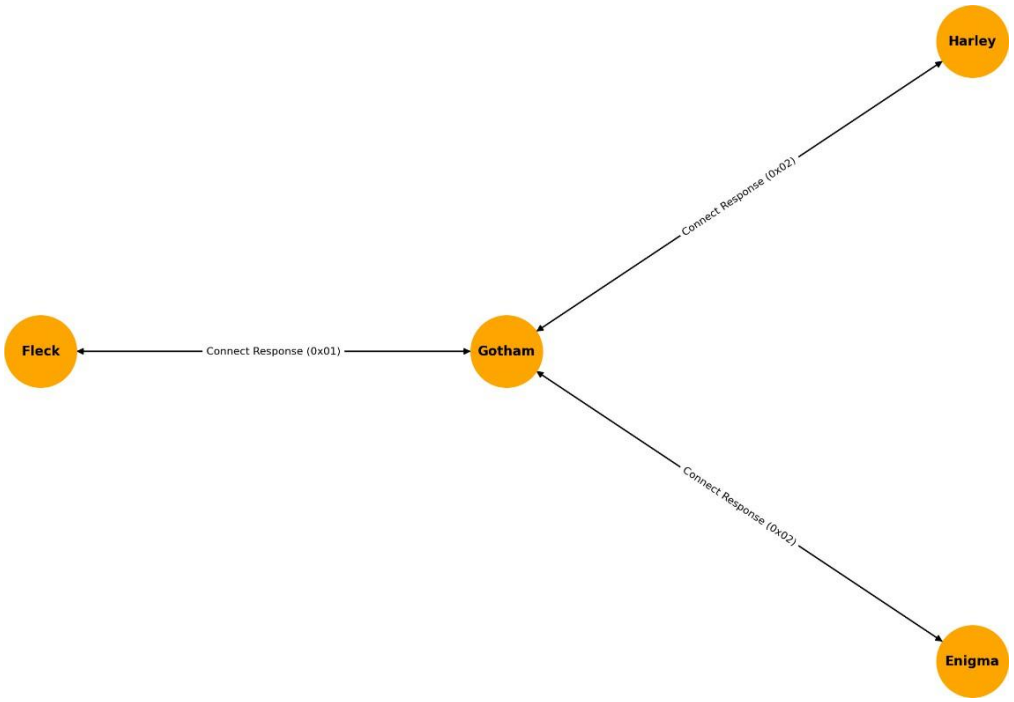


Diagrama que representa el flux d'establiment de connexions entre **Fleck**, **Harley**, **Enigma**, i **Gotham.**

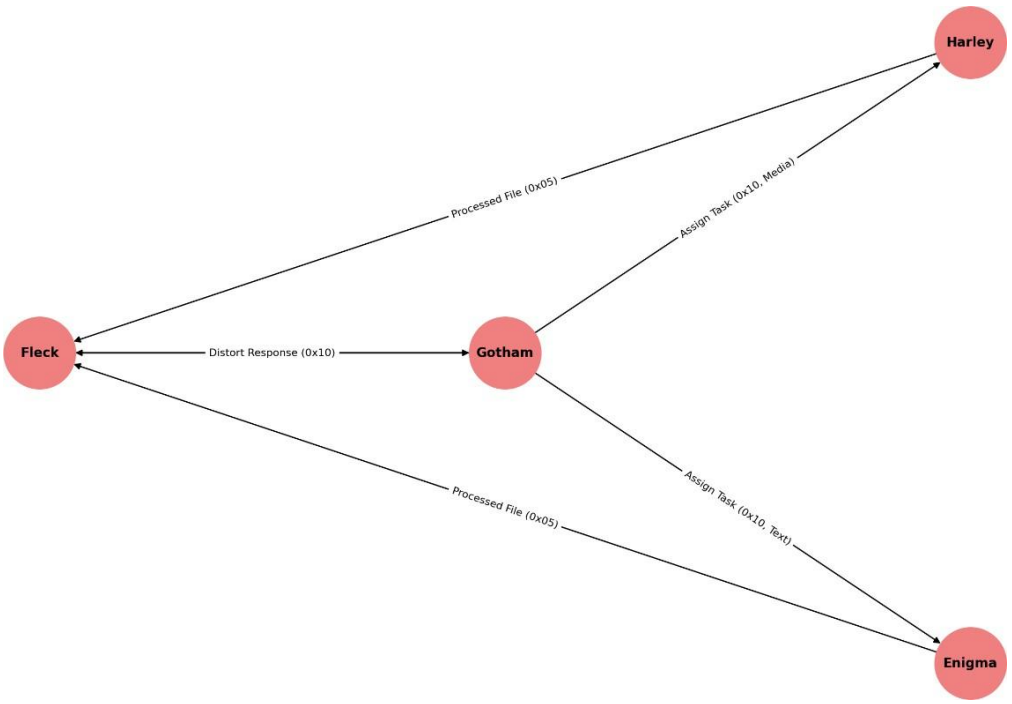
Fleck

1. **Trama de petició**:
    - **Tipus:** 0x01
    - **Contingut:** Inclou informació com el nom de l'usuari, IP, i port del Fleck.
2. **Trama de resposta de Gotham:**
    - **Tipus:** 0x01
    - Si la connexió és exitosa: **Data** està buida (DATA_LENGTH = 0).
    - Si falla: **Data** conté "CON_KO".

Workers

1. **Trama de petició:**
    - **Tipus:** 0x02
    - **Contingut:** Inclou el tipus de worker (e.g., "Media" o "Text"), IP i port del worker.
2. **Trama de resposta de Gotham:**
    - **Tipus:** 0x02
    - Si la connexió és exitosa: **Data** està buida (DATA_LENGTH = 0).
    - Si falla: **Data** conté "CON_KO".

Gestió de Petició de Distorsió



Aquest diagrama mostra el flux d'una petició de distorsió des de **Fleck** cap a **Gotham** i la seva gestió amb els **Workers**.

1. **Petició de Fleck a Gotham**:
    - Trama 0x10 amb informació del fitxer i tipus de distorsió (Media o Text).
2. **Resposta de Gotham a Fleck**:
    - Trama 0x10 amb la informació del worker assignat (IP i port) o missatge d'error (DISTORT_KO o MEDIA_KO).

3. **Assignació de tasques de Gotham als Workers**:
    - Gotham envia la tasca al worker corresponent (Harley per fitxers Media, Enigma per fitxers Text).
4. **Enviament del fitxer processat**:
    - El worker retorna el fitxer distorsionat a Fleck amb trames de tipus 0x05.

6. Estimació temporal

Activitat	Hores dedicades
Investigació	18 h
Disseny	20 h
Implementació	50 h
Testing	20 h
Documentació	12 h
Total	**120 h**

7. Conclusions i propostes de millora

Conclusions sobre la feina feta

Durant el desenvolupament d'aquest projecte, m'he enfrontat a un repte complex que ha posat a prova tant els meus coneixements tècnics com la meva capacitat de gestió personal. El fet d'haver abordat la gran majoria del treball de manera individual ha suposat un esforç considerable, però alhora ha estat una experiència que m'ha motivat a superar dificultats i a buscar solucions creatives per assolir els objectius proposats.

Tot i les limitacions de temps i els imprevistos sorgits durant el procés, considero que el sistema dissenyat compleix, en línies generals, amb la intenció i els requisits principals que se li demanaven. És cert que encara hi ha algunes petites imprecisions i comportaments no del tot polits —com l'execució seqüencial de les comandes des de Fleck— que, amb més temps i recursos, podrien ser clarament millorats i optimitzats.

Tanmateix, valoro molt positivament el recorregut fet. Aquesta experiència m'ha permès posar en pràctica i aprofundir en conceptes clau com l'ús de threads, la memòria compartida, la sincronització amb mutex i condicions, i la programació concurrent en general. Són coneixements que considero essencials en el camp de la informàtica i que, sens dubte, em seran útils en futurs projectes acadèmics o professionals.

En definitiva, malgrat les dificultats, aquest projecte ha estat una oportunitat d'aprenentatge molt enriquidora que m'ha ajudat a créixer tant a nivell tècnic com personal.

Propostes de millora per a la propera entrega

1. **Implementació de la reassignació de tasques interrompudes**
   Es clave para la robustez del sistema, sobre todo en entornos reales donde un worker puede desconectarse o fallar inesperadamente. Tener un registro centralizado y un mecanismo automático para reasignar tareas garantiza continuidad, evita pérdidas y mejora la experiencia del usuario, que podrá ser informado correctamente en caso de interrupciones. Este tipo de gestión mejora la resiliencia y la tolerancia a fallos.

2. **Monitoratge i anàlisi del rendiment en temps real**
   Per tal de mantenir un control constant sobre l'eficiència del sistema i detectar possibles colls d'ampolla o anomalies, es podria implementar un mòdul de monitoratge en temps real que recopili dades de rendiment (temps de processament, ús de CPU, memòria, nombre de connexions actives, etc.) i les presenti mitjançant un dashboard visual o les enviï a un sistema d'alertes. Això facilitaria la identificació proactiva de problemes i permetria ajustar la configuració dels workers o la mida del pool de threads per adaptar-se a les condicions canviants de càrrega.

3. **Cua global de tasques als workers**
   La gestión compartida de una cola de tareas entre workers del mismo tipo en una misma máquina permite balancear la carga, mejorar la eficiencia y reducir la saturación de hilos o procesos individuales. Utilizar memoria compartida y sincronización adecuada con mutex y semáforos es una solución muy escalable que permitirá un mejor aprovechamiento de recursos, sobre todo cuando el sistema crezca en volumen y concurrencia.

4. **Optimització del processament asíncron de Fleck per millorar la concurrència**
   Actualment, Fleck ha d'esperar que finalitzi una distorsió abans de poder començar una altra comanda, cosa que limita la seva capacitat d'execució concurrent i provoca un funcionament gairebé seqüencial. Per millorar aquesta limitació, seria ideal redissenyar Fleck per suportar un model realment asíncron, on les comandes es puguin processar en paral·lel mitjançant l'ús de threads o d'un sistema de cues i esdeveniments. Aquesta millora, encara no implementada per falta de temps i detecció tardana del problema, permetria maximitzar el rendiment i l'eficiència del sistema en situacions amb múltiples peticions simultànies.

8. Bibliografia

- Canaleta, X. (2020). *Guia d'Estil de Sistemes Operatius*. La Salle, Universitat Ramon Llull.
- Canaleta, X. (2020). *Pràctica SO 2024-25*. La Salle, Universitat Ramon Llull.
- Canaleta, X. (2020). *Documents de testing (Fases 1 a 4)*. La Salle, Universitat Ramon Llull.
- Canaleta, X. (2020). *Capítol 4.5.1: Semàfors*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 4.1: Pipes i Memòria Compartida*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 4.2: Cues de Missatges*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 5.1: File Descriptors i Senyals*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2021). *Tema 5.2: Forks*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 5.3: Threads*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 5.4: Sockets*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2021). *Tema 5.5: Pipes*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2020). *Tema 5.6: Memòria Compartida*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.
- Canaleta, X. (2021). *Tema 5.7: Cues de Missatges*. La Salle, Universitat Ramon Llull. Disponible al portal Estudy.